Этап 1

Сущность	Операции	Характер нагрузки	Хранилище	Обоснование	Спорные моменты
1. Пользователи	завести аккаунт, войти, поменять профиль, выдать права	читают и пишут поровну, нужны транзакции	PostgreSQL	аккаунты и роли должны быть целостными, это умеет реляционка	нет
2. Сессии и токены	выдать токен, проверить, снести по сроку	читают на каждый запрос, живут недолго	Redis	мгновенный доступ и срок жизни из коробки	можно и в PostgreSQL, но тормознее
3. Курсы и уроки	автор собирает курс, правит, публикует	читают часто, меняют редко	PostgreSQL с JSONB	вложенность курс-модули-уроки нормально ложится в JSON	часть можно вынести в MongoDB ради гибких полей
4. Прогресс	отметить урок, сохранить тест	постоянно пишут и читают текущее состояние	PostgreSQL	отметку нельзя потерять, нужны гарантии	историю можно слить в ClickHouse
5. Поиск	найти по названию и описанию, фильтры	только чтение и очень много	Elasticsearch	понимает опечатки и ранжирует, обычная база так	придётся синхронизировать с основной базой

				не может	
6. Форум	открыть тему, ответить, лайкнуть	читают и пишут, объём средний	PostgreSQL	дерево ответов и связи с юзерами легко сделать связями	нет
7. Логи действий	клики, просмотры, старт и финиш теста	пишут потоком, читают редко	ClickHouse	глотает огромный поток записи и дёшево хранит	перед ним можно поставить Kafka как буфер
8. Аналитика	сколько дошли за месяц, средний процент, воронка	тяжёлые подсчёты по истории	ClickHouse	колоночная структура быстро считает по большим данным	цифры приезжают с небольшим опозданием, и это ок
9. Рекомендации	похожие курсы, что зашло похожим юзерам	читают плюс периодический пересчёт	PostgreSQL, позже Neo4j	связи юзеров и курсов это по сути граф	отдельная графовая база на старте лишняя

Этап 2

Сущность	**Часто читают или пишут?**	**Нужны транзакции?**	**Нужны JOIN/связи?**	**Объём данных?**	**Требования к задержке?**	**Гибкость схемы?**
1. Пользователи	и то и то	да	да	средний	обычная	нет
2. Сессии и токены	читают	нет	нет	большой	низкая	нет
3. Курсы и уроки	читают	иногда	да	средний	обычная	да
4. Прогресс по курсам	и то и то	да	нет	огромный	низкая	нет
5. Поиск по курсам	читают	нет	нет	средний	низкая	да
6. Форум / обсуждения	и то и то	да	да	средний	обычная	нет
7. Логи действий	пишут	нет	нет	огромный	не важно	да
8. Аналитика и отчёты	читают	нет	да	огромный	не важно	нет
9. Рекомендации курсов	читают	нет	да	средний	не важно	да

- Где данные создаются впервые: пользователи, курсы, прогресс и форум рождаются в основной базе, логи сразу летят в аналитическое хранилище, а поиск и кэш это просто копии основных данных для ускорения

- Какие данные можно кэшировать и с каким TTL: сессии держат в кэше сутки, популярные курсы пару часов, поиск минут 10, рекомендации полдня, а прогресс и права лучше не трогать чтобы не показать старьё
- Какие данные нужны для аналитики и те же это данные или производные: аналитике нужны производные, она берёт сырые логи и прогресс и считает из них общее количество прошедших курс или среднее время прохождения
- Где нужны ACID-гарантии и где хватит eventual consistency: строгие гарантии нужны пользователям, прогрессу и форуму чтобы не потерять отметку о прохождении или ответ, а поиску и аналитике нормально если данные обновятся через пару секунд

Этап 3

Сущность / поток	**Операции**	**Характер нагрузки**	**Хранилище**	**Обоснование**	**Источник истины?**	**Кэш?**	**Спорные моменты**
1. Пользователи	регистрация, вход, правка профиля, права	читают и пишут поровну, транзакции	PostgreSQL	аккаунты и роли не должны разъехаться, реляционка это умеет	да	да, профиль около часа	нет
2. Сессии и токены	выдать токен, проверить, снести по сроку	читают на каждый запрос, живут недолго	Redis	мгновенный доступ и встроенный срок жизни, транзак	да, это временные данные	нет, это и есть кэш	если Redis умрёт, все вылогинятся

				ции не нужны			
3. Курсы и уроки	автор собирает и публикует, студенты читают	читают часто, пишут редко	PostgreSQL + Redis + Elasticsearch	вложенность ложится в JSONB, частые чтения надо кэшировать и искать отдельно	да, PostgreSQL	да, популярные курсы пару часов	копии в двух местах, нужна синхронизация
4. Прогресс	отметить урок, сохранить тест	постоянная запись, частое чтение текущего	PostgreSQL + ClickHouse	отметку нельзя потерять поэтому гарантии, история уходит в аналитику	да, PostgreSQL	осторожно, на пару минут	дублирование в ClickHouse после коммита
5. Поиск	найти по названию и описанию, фильтры	только чтение, очень много	Elasticsearch, копия из PostgreSQL	понимает опечатки и ранжирует, обычная база	нет, производные	да, частые запросы минут на 10	индекс отстаёт на пару секунд

				так не может			
6. Форум	тема, ответы, лайки	читают и пишут, объём средний	PostgreSQL	дерево ответов и связи с юзерами легко сделать связями, нужны транзакции	да	да, топ темы на пару минут	нет
7. Логи действий	клики, просмотры, старт и финиш теста	пишут потоком, читают редко	ClickHouse, опционально Kafka перед ним	глотает огромный поток и дёшево хранит, не нагружает основную базу	да для логов	нет	нужна ли Kafka на старте, спорно
8. Аналитика и отчёты	сколько дошли за месяц, средний процент, воронка	тяжёлые агрегаты по истории	ClickHouse	быстро считает по большим данным и не мешает основной базе	нет, производные от логов и прогресса	да, готовые отчёты на пару часов	цифры приезжают с опозданием
9.	похож	чтение	Postgre	связи	нет,	да,	когда

Рекомендации	ие курсы, что зашло похожим юзерам	плюс периодический пересчёт	SQL сейчас, Neo4j позже	юзеров и курсов это по сути граф, но на старте хватит реляционки	производные	результат на полдня	пора тащить графовую базу, непонятно

Спорные моменты отдельным списком

- Сессии живут только в Redis, и если он упадёт, все пользователи вылогинятся, можно дублировать в PostgreSQL.
- Курсы лежат в трёх местах сразу, поэтому поиск и кэш могут короткое время показывать устаревшее.
- Прогресс дублируется в ClickHouse, и нужен механизм чтобы отметки не терялись по дороге.
- Kafka перед ClickHouse на старте не обязательна, можно писать напрямую.
- Рекомендации пока в PostgreSQL, и нет чёткого критерия когда пора переезжать на графовую базу.

Самопроверка

1. В Redis лежат только сессии и кэш, и транзакции там не нужны, так что нарушения нет.
2. Логи в PostgreSQL не лежат и уходят сразу в ClickHouse, поэтому основная база не растёт без лимита.
3. Тяжёлые агрегаты идут только в ClickHouse и не трогают OLTP базу.
4. У всех сущностей в нескольких хранилищах синхронизация описана, значит карта сходится.

Этап 4